

Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Curso: Algoritmos y Estructuras de Datos I CE-1103

LogísTEC

Profesor: MSc. Andrés Vargas Rivera

Estudiantes:

Alejandro Arias López 2025074700
Andrés Emiliano Aguilar Méndez 2025080405
Emmanuel Martín Rojas Navarro 2025143457
José Pablo Valverde Durán 2025088421

13 de junio de 2026

Contents

1	Manual de Usuario	3
1.1	Software y Hardware mínimos para la ejecución	3
1.2	Compilación	3
1.3	Ejecución	3
1.4	Formato que debe tener el JSON	4
2	Bibliotecas de Terceros	4
3	Métodos Implementados por Paquete	4
3.1	Paquete <code>graph</code>	4
3.2	Paquete <code>util</code>	6
3.3	Paquete <code>algorithms</code>	7
3.4	Paquete <code>planner</code>	9
3.5	Paquetes <code>io</code> y <code>ui</code>	9
4	Estructuras de Datos Desarrolladas	10
4.1	Lista Enlazada Simple — <code>LinkedList<T></code>	10
4.2	Cola de Prioridad Mínima — <code>MinHeap<T></code>	10
4.3	Conjunto Disjunto — <code>UnionFind</code>	10
4.4	TDA Grafo — <code>Graph</code>	11
5	Algoritmos Implementados	11
5.1	BFS — Búsqueda en Anchura	12
5.2	DFS — Búsqueda en Profundidad	12
5.3	Warshall — Cierre Transitivo	13
5.4	Dijkstra — Camino Mínimo desde un Origen	13
5.5	Floyd-Warshall — Todos los Pares	14
5.6	Prim — Árbol de Expansión Mínima	15
5.7	Kruskal — Árbol de Expansión Mínima	16
5.8	Nearest Neighbor — Heurística de Ruteo	17
5.9	MST-based — Heurística 2-Aproximación	18

6	Comparaciones Empíricas	19
6.1	Prim vs Kruskal	19
6.2	Nearest Neighbor vs MST-based	19
7	Problemas Conocidos y Limitaciones	20
8	Actividades por Estudiante	20
9	Problemas Encontrados	21
9.1	Conflicto de nombre: Package vs java.lang.Package	21
9.2	Desbordamiento aritmético en Floyd-Warshall	21
10	Conclusiones y Recomendaciones	22
11	Bibliografía	22
12	Diagrama de Clases — Justificación del Diseño	22
12.1	Organización en paquetes	23
12.2	Principios de diseño aplicados	23
13	Bitácora de Trabajo	23

1 Manual de Usuario

1.1 Software y Hardware mínimos para la ejecución

Componente	Requisito mínimo para ejecutar
Java	JDK 17 LTS o superior (probado con OpenJDK 21)
Sistema operativo	Windows 10+, Ubuntu 20.04+, macOS 12+
RAM	256 MB disponibles para la JVM
Espacio en disco	5 MB (binarios + datos de prueba)
Pantalla	Resolución mínima 1280×800 (para la GUI)

1.2 Compilación

Para compilar este proyecto se utiliza el script de `build.sh` que se encuentra dentro del directorio `logistec/`. Lo que este script hace es detectar todos los archivos necesarios (los `.java`), los compila hacia `out/`, y el resultado es empaquetado en el archivo `logistec.jar`.

```
# Situar en el directorio raíz del proyecto
cd logistec/
# Compilar y generar el JAR
bash build.sh
```

1.3 Ejecución

Modo GUI (interfaz gráfica Swing)

```
java -jar logistec.jar
# o bien:
bash build.sh run
```

La ventana principal permite: cargar un archivo JSON con el botón *Cargar Caso*, ejecutar todos los algoritmos, consultar caminos mínimos por par de vértices, y visualizar las rutas de cada camión con colores distintos.

Modo texto (headless)

```
java -jar logistec.jar data/caso_prueba.json
# o bien:
bash build.sh test
```

Imprime en consola el reporte completo: matriz de Warshall, asignación de paquetes, costo del MST (Prim y Kruskal), matriz Floyd-Warshall y rutas de camiones.

1.4 Formato que debe tener el JSON

```
{
  "ciudad": {
    "vertices": [
      {"id": "D", "tipo": "DEPOT", "x": 500, "y": 350},
      {"id": "A", "tipo": "INTERSECCION", "x": 100, "y": 100},
      {"id": "BB", "tipo": "DELIVERY", "x": 380, "y": 300}
    ],
    "aristas": [
      {"u": "D", "v": "BB", "distancia": 220},
      {"u": "D", "v": "A", "distancia": 450}
    ]
  },
  "paquetes": [
    {"id": "P01", "destino": "BB", "peso": 5, "prioridad": 1}
  ],
  "camiones": [
    {"id": "C01", "capacidad": 50},
    {"id": "C02", "capacidad": 30}
  ]
}
```

2 Bibliotecas de Terceros

Dentro de este proyecto no hay dependencias de terceros. En el proyecto se usa un parser JSON propio (`JsonParser.java`), se emplea mediante un analizador sintáctico recursivo descendente. No se hace uso de ninguna librería externa. Las APIs que se usan de la JDK son usadas en Strings y arreglos primitivos (`java.io`, `java.nio`).

Componente	Fuente	Para qué se usó
<code>javax.swing.*</code>	JDK estándar	Interfaz gráfica (GUI)
<code>java.awt.*</code>	JDK estándar	Renderizado 2D del grafo
<code>java.io.*</code>	JDK estándar	Lectura del archivo JSON
<code>java.nio.file.*</code>	JDK estándar	Lectura eficiente de archivos

3 Métodos Implementados por Paquete

3.1 Paquete graph

`Vertex.java` — Nodo del grafo

Campo	Tipo	Descripción
id (vértice)	String	Identificador único del vértice
Tipo	String	DEPOT, INTERSECCION o DELIVERY
x, y	int	Coordenadas de pantalla en píxeles
u, v	String	IDs de los extremos de la arista
Distancia	int	Peso de la arista en metros (entero positivo)
id (paquete)	String	Identificador único del paquete
Destino	String	ID del vértice de entrega
Peso	int	Peso del paquete en kg
Prioridad	int	1 es máxima prioridad y 3 es la mínima
id (camión)	String	Identificador del camión
Capacidad	int	Máximo peso que puede soportar cada camión

Método/Constructor	Descripción
Vertex(id, type, x, y)	Constructor principal
String getId()	Método usado para saber el ID del vértice
Type getType()	Retorna el tipo (DEPOT / INTERSECTION / DELIVERY)
int getX(), getY()	Coordenadas de pantalla
boolean isDepot()	Método usado para saber DEPOT (Retorna True en caso de que sí)
boolean isDelivery()	Método usado para saber DELIVERY (Retorna True en caso de que sí)

Edge.java — Arista no dirigida y ponderada

Método	Descripción
Edge(u, v, weight)	Constructor
int getU(), getV()	Extremos de la arista (índices de vértices)
int getWeight()	Peso en metros
int other(int vertex)	Retorna el otro extremo dado uno de ellos
int compareTo(Edge o)	Ordenamiento natural por peso (usado por Kruskal)

Graph.java — TDA Grafo no dirigido ponderado

Método	Descripción
Graph(int V)	Crea un grafo con V vértices
void addVertex(index, v)	Inserta un vértice en la posición dada

Método	Descripción
<code>void addEdge(u, v, weight)</code>	Agrega arista no dirigida; actualiza lista de aristas
<code>int indexOf(String id)</code>	Búsqueda de índice por id — $O(V)$
<code>int numVertices(), numEdges()</code>	Tamaño del grafo
<code>int getDepotIndex()</code>	Índice del único depósito
<code>LinkedList<AdjEntry> adj(v)</code>	Lista de adyacencia del vértice v
<code>LinkedList<Edge> getAllEdges()</code>	Todas las aristas (para Kruskal)
<code>int[][] adjacencyMatrix()</code>	Matriz de pesos $V \times V$ (para Floyd-Warshall)
<code>boolean[][] booleanMatrix()</code>	Matriz booleana de adyacencia (para Warshall)

Parcel.java — Paquete logístico

Método	Descripción
<code>Parcel(id, destinationId, weight, priority)</code>	Constructor
<code>void assign(truckId)</code>	Marca el paquete como asignado
<code>void reject()</code>	Marca el paquete como rechazado
<code>Status getStatus()</code>	Estado: PENDING / ASSIGNED / REJECTED
<code>int getDestinationIndex()</code>	Índice del vértice de destino

Truck.java — Camión de la flota

Método	Descripción
<code>boolean addPackage(Parcel p)</code>	Intenta agregar el paquete; false si no cabe
<code>boolean canFit(Parcel p)</code>	Verifica si el paquete cabe en la capacidad libre
<code>int getFreeCapacity()</code>	Capacidad disponible actual en kg
<code>double occupancyPercent()</code>	Porcentaje de ocupación
<code>setRoute(LinkedList<Integer>)</code>	Asigna la ruta de paradas
<code>setRouteDistanceNN(int)</code>	Almacena distancia de heurística NN
<code>setRouteDistanceMST(int)</code>	Almacena distancia de heurística MST-based

3.2 Paquete util

LinkedList<T>.java

Método	Complejidad	Descripción
<code>addLast(item)</code>	$O(1)$	Agrega al final
<code>addFirst(item)</code>	$O(1)$	Agrega al inicio
<code>removeFirst()</code>	$O(1)$	Elimina y retorna el primero
<code>peekFirst()</code>	$O(1)$	Consulta el primero sin eliminar
<code>get(int i)</code>	$O(n)$	Retorna el elemento en posición i
<code>set(int i, T item)</code>	$O(n)$	Modifica el elemento en posición i
<code>remove(int i)</code>	$O(n)$	Elimina el elemento en posición i
<code>contains(T item)</code>	$O(n)$	Verifica pertenencia
<code>size()</code>	$O(1)$	Cantidad de elementos
<code>toArray()</code>	$O(n)$	Snapshot como arreglo

MinHeap<T>.java — Cola de prioridad mínima

Método	Complejidad	Descripción
<code>insert(item, key)</code>	$O(\log n)$	Inserta con prioridad dada
<code>extractMin()</code>	$O(\log n)$	Extrae el elemento de menor clave
<code>peekMin()</code>	$O(1)$	Consulta el mínimo sin extraer
<code>decreaseKey(item, newKey)</code>	$O(n + \log n)$	Reduce la clave de un elemento existente
<code>contains(item)</code>	$O(n)$	Verifica si el item está en el heap

Queue<T>.java y Stack<T>.java

Operación	Queue	Stack	Complejidad
Insertar	<code>enqueue()</code>	<code>push()</code>	$O(1)$
Extraer	<code>dequeue()</code>	<code>pop()</code>	$O(1)$
Consultar	<code>peek()</code>	<code>peek()</code>	$O(1)$

UnionFind.java — Conjunto disjunto

Método	Complejidad	Descripción
<code>find(x)</code>	$O(\alpha(n))$	Representante del conjunto de x
<code>union(x, y)</code>	$O(\alpha(n))$	Une conjuntos; retorna true si fusionó
<code>connected(x, y)</code>	$O(\alpha(n))$	Verifica si x e y están en el mismo conjunto
<code>components()</code>	$O(1)$	Número de componentes distintas

3.3 Paquete algorithms

Clase	Método	Descripción
GraphTraversal	bfs(g, src)	Búsqueda en anchura; retorna lista de vértices visitados
	dfs(g, src)	Búsqueda en profundidad (iterativa con pila)
	dfsPreorder(g, src, visited, order)	DFS preorden para recorrer MST
	connectedComponents(g)	Detecta componentes conexas; retorna número
Warshall	Constructor(g)	Ejecuta cierre transitivo en tiempo $O(V^3)$
	canReach(src, dst)	Retorna true si existe camino de src a dst
	getMatrix()	Retorna matriz booleana P donde $P[i][j]$ = alcanzable
Dijkstra	Constructor(g, src)	Ejecuta desde src en $O((V+E) \log V)$
	distanceTo(dst)	Retorna distancia mínima a dst
	pathTo(dst)	Retorna LinkedList con el camino completo
	hasPath(dst)	Verifica alcanzabilidad desde src
Floyd-Warshall	Constructor(g)	Calcula todos los pares en $O(V^3)$
	dist(i, j)	Distancia mínima entre vértices i y j
	reconstructPath(i, j)	Retorna camino completo $i \rightarrow j$
	getDistMatrix()	Retorna matriz de distancias $\text{int}[][]$
Prim	Constructor(g)	Ejecuta MST desde vértice 0 en $O((V+E) \log V)$
	getMSTEdges()	Retorna $\text{LinkedList}<\text{Edge}>$ con aristas del árbol
	getTotalWeight()	Costo total del MST
	getElapsedMs()	Tiempo de ejecución en milisegundos
Kruskal	Constructor(g)	Ejecuta MST por ordenamiento en $O(E \log E)$
	getMSTEdges()	Retorna $\text{LinkedList}<\text{Edge}>$ con aristas del árbol
	getTotalWeight()	Costo total del MST (debe igualar Prim)
	getElapsedMs()	Tiempo de ejecución en milisegundos

3.4 Paquete planner

Método	Descripción
<code>validateReachability(packages)</code>	Marca como REJECTED los paquetes con destino inalcanzable (usa matriz de Warshall). Retorna número de rechazados.
<code>assignPackages(packages, trucks)</code>	Asignación Best-Fit: 1. Ordena paquetes por prioridad ASC, luego peso DESC. 2. Para cada paquete, asigna al camión con mayor capacidad libre que quepa. 3. Marca como REJECTED si ningún camión puede llevar. Retorna número de paquetes asignados exitosamente.
<code>planRoutes(trucks)</code>	Para cada camión con paquetes: 1. Calcula ruta Nearest Neighbor 2. Calcula ruta MST-based 3. Asigna la ruta de menor distancia a cada camión
<code>nearestNeighbor(int[] stops)</code>	Heurística voraz $O(n^2)$: Parte del depósito, siempre va a parada no visitada más cercana, repite, retorna al depósito. Retorna <code>int[]</code> con el orden de visita (incluyendo depósito).
<code>mstBased(int[] stops)</code>	Heurística 2-aproximación $O(n^2 + n \log n)$: 1. Construye MST del subgrafo $\{\text{depósito} \cup \text{paradas}\}$ 2. Realiza DFS preorden desde depósito 3. Retorna <code>int[]</code> con orden de aparición (garantía $\leq 2 \times \text{óptimo}$)
<code>routeDistance(int[] route)</code>	Calcula distancia total de una ruta sumando distancias Floyd-Warshall entre vértices consecutivos. Retorna <code>int</code> con distancia total en metros.

3.5 Paquetes io y ui

Método	Descripción
<code>JsonParser</code>	Parser JSON recursivo descendente propio. Expone <code>JsonObject</code> y <code>JsonArray</code> .
<code>CityLoader</code>	Construye <code>Graph</code> , <code>Parcel[]</code> y <code>Truck[]</code> desde el JSON; resuelve índices.

Método	Descripción
ReportGenerator	Genera el reporte en texto plano con todas las secciones del proyecto.
MainWindow	Ventana principal Swing: barra, panel de grafo, panel de reporte, controles.
GraphPanel	Extiende JPanel; dibuja vértices, aristas, MST y rutas de camiones.

4 Estructuras de Datos Desarrolladas

4.1 Lista Enlazada Simple — `LinkedList<T>`

Representación interna: nodos con puntero `next`; referencias `head` y `tail`; contador de tamaño. Implementa `Iterable<T>` para uso con `for-each`.

Operación	Complejidad	Notas
<code>addFirst</code> / <code>addLast</code>	$O(1)$	Acceso directo a <code>head</code> / <code>tail</code>
<code>removeFirst</code>	$O(1)$	—
<code>get(i)</code> / <code>set(i)</code>	$O(n)$	Recorre la cadena
<code>contains</code> / <code>remove(T)</code>	$O(n)$	Búsqueda lineal
<code>size()</code>	$O(1)$	Campo contador mantenido

4.2 Cola de Prioridad Mínima — `MinHeap<T>`

Representación interna: arreglo dinámico de entradas `Entry<T>(item, key)`. El arreglo se duplica cuando se llena (*grow*). Invariante: `key[parent] ≤ key[child]` para todo nodo.

- `siftUp(i)`: intercambia el nodo con su padre mientras sea menor — $O(\log n)$.
- `siftDown(i)`: intercambia con el hijo menor mientras sea mayor — $O(\log n)$.
- `decreaseKey(item, newKey)`: búsqueda lineal + `siftUp` — $O(n + \log n)$.

4.3 Conjunto Disjunto — `UnionFind`

Representación: arreglos `parent[]` y `rank[]`. Optimizaciones: compresión de caminos (en `find`, todos los nodos apuntan a la raíz) y unión por rango (árbol de menor altura se adjunta al mayor). Complejidad amortizada $O(\alpha(n)) \approx O(1)$ para todas las operaciones.

4.4 TDA Grafo — Graph

Representación: lista de adyacencia mediante `LinkedList<AdjEntry>[]`. También se mantiene una `LinkedList<Edge>` con todas las aristas (requerida por Kruskal). La clase interna `AdjEntry` guarda el par (`target: int, weight: int`).

Operación	Complejidad	Descripción
<code>addVertex</code>	$O(1)$	Inserción directa por índice
<code>addEdge</code>	$O(1)$	Dos <code>addLast</code> en <code>adj</code> + 1 en <code>edges</code>
<code>indexOf(id)</code>	$O(V)$	Búsqueda lineal por id
<code>adj(v)</code>	$O(1)$	Retorna la lista del vértice v
<code>adjacencyMatrix()</code>	$O(V^2 + E)$	Construye matriz densa
<code>booleanMatrix()</code>	$O(V^2 + E)$	Construye matriz booleana

5 Algoritmos Implementados

Para todos los ejemplos se usa el grafo de referencia de 5 vértices:

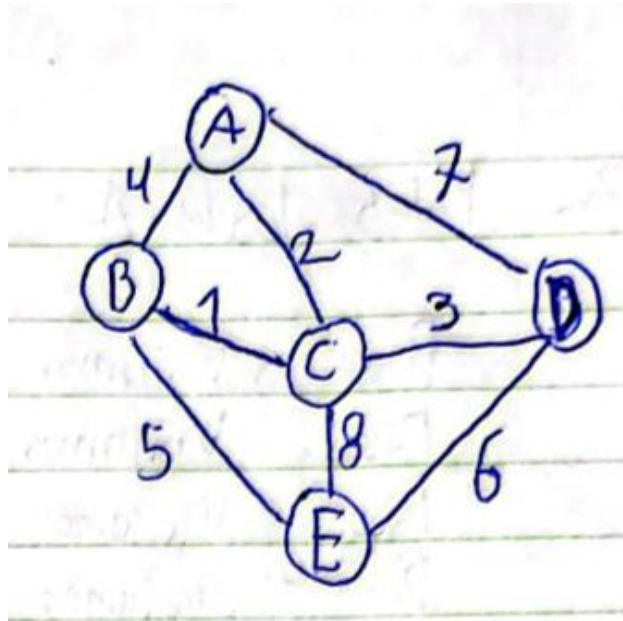


Figure 1: Grafo ponderado no dirigido de 5 vértices

$$V = \{A, B, C, D, E\}, \quad \text{Depósito} = A$$

$$E = \{(A-B, 4), (A-C, 2), (A-D, 7), (B-C, 1), (C-D, 3), (B-E, 5), (C-E, 8), (D-E, 6)\}$$

5.1 BFS — Búsqueda en Anchura

Complejidad: $O(V + E)$

```
BFS(G, src):
    visited[0..V-1] = false
    cola = Queue(); cola.enqueue(src); visited[src] = true
    mientras cola no vacía:
        u = cola.dequeue()
        para cada vecino v de u:
            si no visited[v]: visited[v] = true; cola.enqueue(v)
```

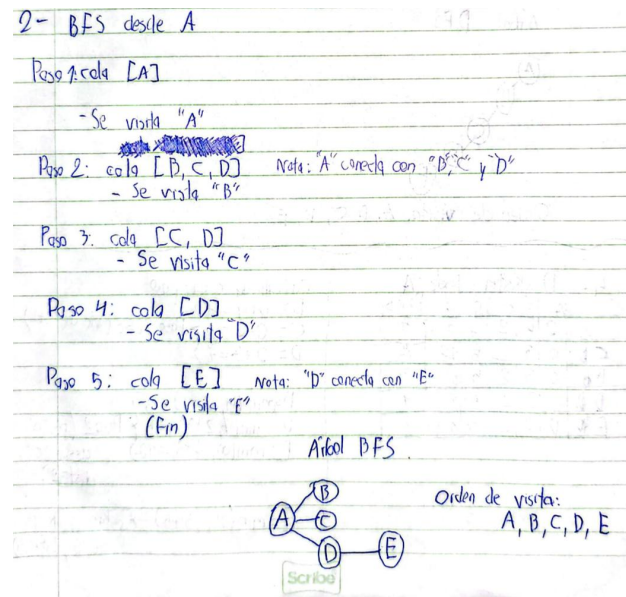


Figure 2: Ejemplo de BFS

5.2 DFS — Búsqueda en Profundidad

Complejidad: $O(V + E)$

```
DFS(G, src):
    visited[0..V-1] = false
    pila = Stack(); pila.push(src)
    mientras pila no vacía:
        u = pila.pop()
        si visited[u]: continuar
        visited[u] = true
        para cada vecino v de u (en orden inverso): pila.push(v)
```

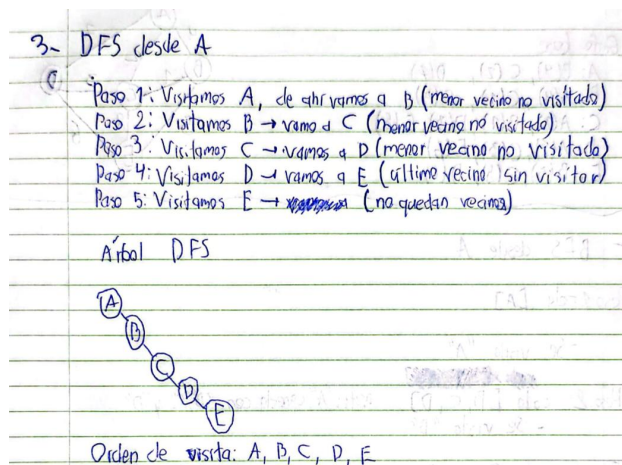


Figure 3: Ejemplo de DFS

5.3 Warshall — Cierre Transitivo

Complejidad: $O(V^3)$

Warshall(G):

```
P = booleanMatrix(G) // P[i][i] = true
para k = 0..V-1:
  para i = 0..V-1:
    para j = 0..V-1:
      P[i][j] = P[i][j] OR (P[i][k] AND P[k][j])
retornar P
```

Ejemplo: el grafo de referencia es conexo, por tanto $P[i][j] = \text{true}$ para todo par. Se forma una matriz donde todos sus valores son 1.

5.4 Dijkstra — Camino Mínimo desde un Origen

Complejidad: $O((V+E) \log V)$ con cola de prioridad

Dijkstra(G, src):

```
dist[0..V-1] = INF; dist[src] = 0
pq = MinHeap(); pq.insert(src, 0)
mientras pq no vacía:
  u = pq.extractMin()
  para cada arista (u, v, w) en adj(u):
    alt = dist[u] + w
    si alt < dist[v]:
      dist[v] = alt; prev[v] = u
      pq.decreaseKey(v, alt)
```

4- Dijkstra desde A

	A	B	C	D	E	Pesos
A	0	4	2	∞		7
C	0	3	2	5	10	2
B	0	3	2	5	8	3
D	0	3	2	5	8	4
E	0	3	2	5	8	5

Cálculo de cada paso

$$B = 0 + 4 = 4$$

$$C = 0 + 2 = 2 \text{ Paso 1 (se parte de A)}$$

$$D = 0 + 7 = 7$$

$$D_{\min}(4, 2+1) = 3$$

$$D = \min(7, 2+3) = 5 \text{ Paso 2 (parte desde C, dist=2)}$$

$$E = \min(\infty, 2+8) = 10$$

$$E = \min(10, 3+5) = 8 \text{ Paso 3 (parte de B, dist=3)}$$

Scrabe

Figure 4: Ejemplo de Dijkstra

5.5 Floyd-Warshall — Todos los Pares

Complejidad: $O(V^3)$

```

FloydWarshall(G):
    D = adjacencyMatrix(G) // D[i][i]=0, D[i][j]=peso o INF
    next[i][j] = j si arista, sino -1
    para k = 0..V-1:
        para i = 0..V-1:
            si D[i][k] == INF: continuar
            para j = 0..V-1:
                si D[i][k] + D[k][j] < D[i][j]:
                    D[i][j] = D[i][k] + D[k][j]
                    next[i][j] = next[i][k]
    retornar D, next
  
```

5 Floyd - Warshall

Matriz inicial (cero)

	A	B	C	D	E
A	0	4	2	7	∞
B	4	0	1	∞	5
C	2	1	0	3	8
D	7	∞	3	0	6
E	∞	5	8	6	0

$K \rightarrow K=C$

	A	B	C	D	E
A	0	3	2	5	8
B	3	0	1	4	5
C	2	1	0	3	6
D	5	4	3	0	6
E	8	5	6	6	0

$K=A$

	A	B	C	D	E
A	0	4	2	7	∞
B	4	0	1	11	5
C	2	1	0	3	8
D	7	11	3	0	6
E	∞	5	8	6	0

$K=D$

	A	B	C	D	E
A	0	3	2	5	8
B	3	0	1	4	5
C	2	1	0	3	6
D	5	4	3	0	6
E	8	5	6	6	0

$K=B$

	A	B	C	D	E
A	0	4	2	7	9
B	4	0	1	11	5
C	2	1	0	3	6
D	7	11	3	0	6
E	9	5	6	6	0

$K=E$ (matriz final)

	A	B	C	D	E
A	0	3	2	5	8
B	3	0	1	4	5
C	2	1	0	3	6
D	5	4	3	0	6
E	8	5	6	6	0

Figure 5: Ejemplo de Floyd-Warshall

5.6 Prim — Árbol de Expansión Mínima

Complejidad: $O((V+E) \log V)$

Prim(G):

```

key[0..V-1] = INF; key[0] = 0; inTree[0..V-1] = false
pq = MinHeap(); para i en V: pq.insert(i, key[i])
mientras pq no vacía:
    u = pq.extractMin(); inTree[u] = true
    si parent[u] != -1: añadir arista (parent[u], u, key[u]) al MST
    para cada arista (u, v, w):
        si no inTree[v] y w < key[v]:
            key[v] = w; parent[v] = u; pq.decreaseKey(v, w)

```

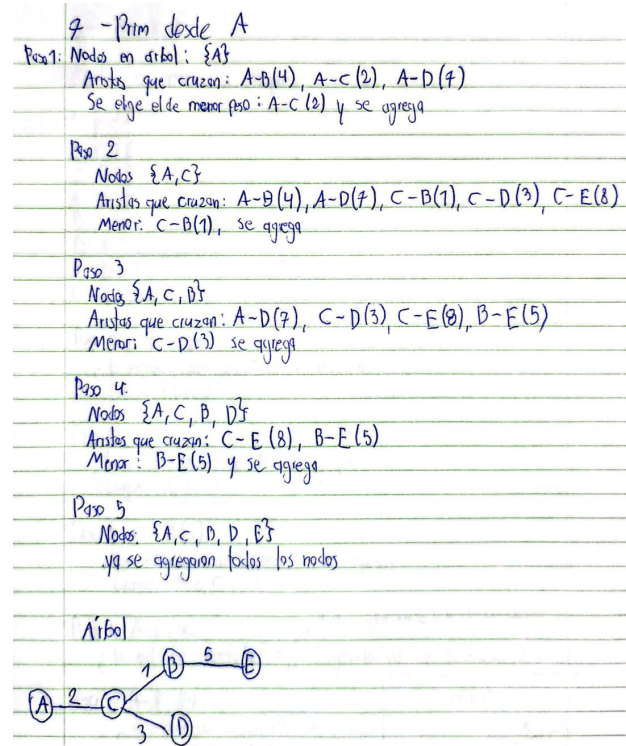



Figure 6: Ejemplo de Prim

5.7 Kruskal — Árbol de Expansión Mínima

Complejidad: $O(E \log E)$, dominado por ordenamiento

Kruskal(G):

aristas = ordenar todas por peso ASC

uf = UnionFind(V)

para cada arista (u, v, w) en aristas:

si uf.union(u, v): // true si une dos componentes distintas

anadir arista al MST

si |MST| == V-1: break

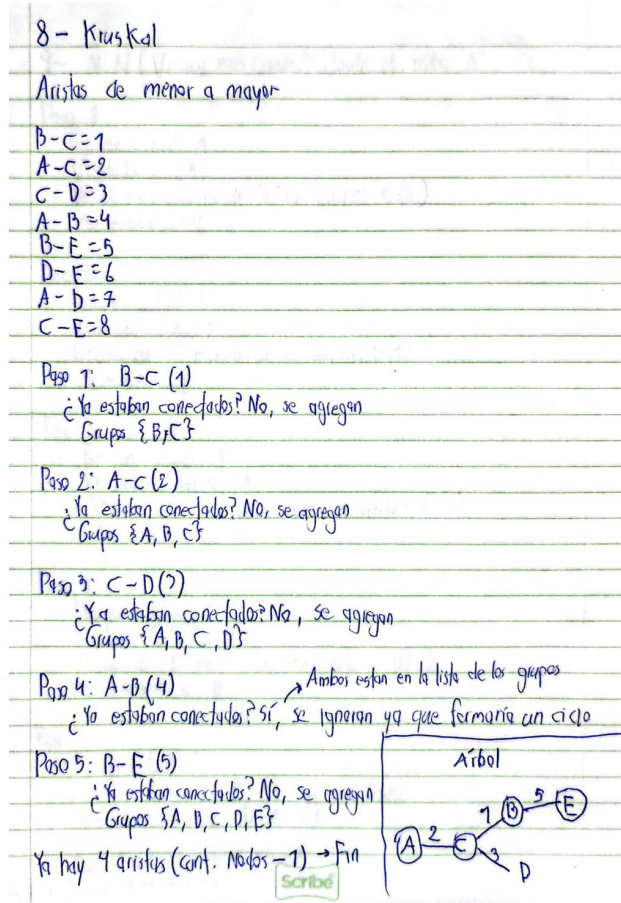


Figure 7: Ejemplo de Kruskal

5.8 Nearest Neighbor — Heurística de Ruteo

Complejidad: $O(n^2)$ donde n = número de paradas

```
NearestNeighbor(depot, stops[], D):
    visited[0..n-1] = false
    ruta = [depot]; cur = depot
    para step = 0..n-1:
        mejor = -1; mejorDist = INF
        para j = 0..n-1:
            si no visited[j] y D[cur][stops[j]] < mejorDist:
                mejorDist = D[cur][stops[j]]; mejor = j
            visited[mejor] = true; cur = stops[mejor]
        ruta.agregar(cur)
    ruta.agregar(depot); retornar ruta
```

9- NN (Vecino más cercano) desde el nodo 'A'

Paso 1
 Posición actual: A
 Visitados: {A}
 Vecino más cercano de 'A' sin visitar: C(2)
 Me muevo a C

Paso 2
 Posición actual: C
 Visitados: {A, C}
 Vecino de 'C' más cerca sin visitar: B(1)
 Me muevo a B

Paso 3
 Posición actual: B
 Visitados: {A, C, B}
 Vecino de 'B' más cercano sin visitar: E(5)
 Nos movemos a E

Paso 4
 Posición actual: E
 Visitados: {A, C, B, E}
 Vecino de 'E' más cercano sin visitar: D(6)
 No movemos a D

Paso 5
 Posición actual: D
 Visitados: {A, C, B, E, D} ya son todos
 Regresamos al inicio: D-A (7)
 Ruta Final: A→C→B→E→D→A
 costo total: $2+1+5+6+7=21$

Figure 8: Ejemplo de Nearest Neighbor

5.9 MST-based — Heurística 2-Aproximación

Complejidad: $O(n^2 + n \log n)$

MSTBased(depot, stops[], D):

```
// 1. Construir subgrafo completo sobre {depot} U {stops}
sub = Graph(n+1); usar D[subV[i]][subV[j]] como pesos
// 2. MST del subgrafo
kruskal = Kruskal(sub); mstGraph = kruskal.buildMSTGraph()
// 3. DFS preorden desde el deposito
preorden = dfsPreorder(mstGraph, 0, visited)
// 4. Mapear a indices del grafo principal y anadir retorno
ruta = [subV[v] para v en preorden] + [depot]
retornar ruta
```

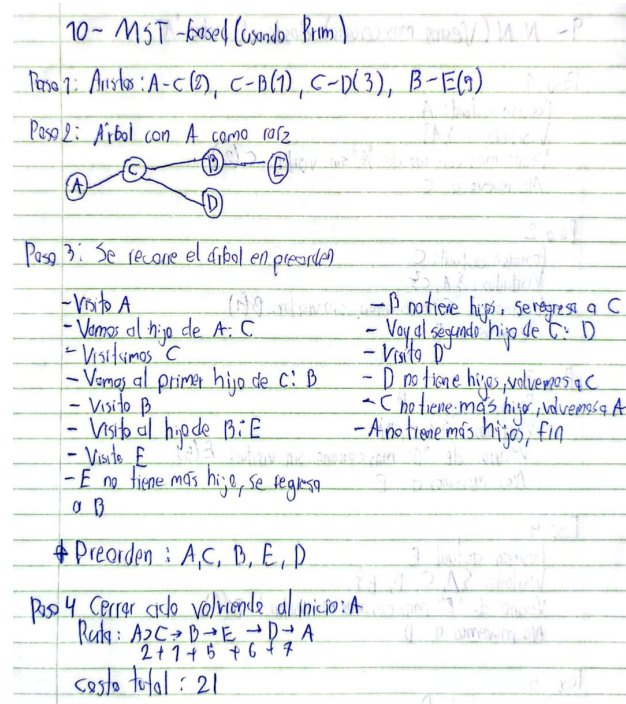


Figure 9: Ejemplo de MST-Based

6 Comparaciones Empíricas

6.1 Prim vs Kruskal

Los tiempos se midieron con `System.nanoTime()` dentro del constructor de cada clase.

Instancia	V	E	Costo MST (metros)	Prim (ms)	Kruskal (ms)
Caso prueba	31	54	4175 m	2.073	0.946
Instancia 2	83	138	8500 m	0.221	0.133
Instancia 3	121	201	12600 m	0.228	0.184

6.2 Nearest Neighbor vs MST-based

Fórmula de mejora: $(d_{NN} - d_{MST})/d_{NN} \times 100\%$

Instancia / Camión	Paradas	Dist. NN (m)	Dist. MST (m)	Mejora (%)
Caso prueba C01	5	2415	2415	0.0%
Caso prueba C02	5	2645	2810	-6.2%
Caso prueba C03	5	2845	2550	10.4%
Instancia 2 C01	4	4280	4280	0.0%

Instancia / Camión	Paradas	Dist. NN (m)	Dist. MST (m)	Mejora (%)
Instancia 2 C02	4	3680	3680	0.0%
Instancia 2 C03	4	3480	3480	0.0%
Instancia 2 C04	2	3440	3440	0.0%
Instancia 2 C05	3	3260	3260	0.0%
Instancia 2 C06	1	1820	1820	0.0%
Instancia 3 C01	7	5480	5680	-3.6%
Instancia 3 C02	7	5680	4880	14.1%
Instancia 3 C03	7	5480	5480	0.0%
Instancia 3 C04	5	4800	4800	0.0%

7 Problemas Conocidos y Limitaciones

1. **Escalabilidad de decreaseKey:** la búsqueda lineal $O(n)$ degrada Dijkstra y Prim de $O((V+E) \log V)$ a $O(E \cdot V)$ para grafos muy densos. En el rango del proyecto ($V \leq 100$, $E \leq 300$) el impacto es insignificante.
2. **Grafo no conexo:** los paquetes con destino en componentes distintas al depósito se rechazan automáticamente. El MST solo cubre la componente del depósito (comportamiento esperado).
3. **Aristas paralelas:** si el JSON incluye más de una arista entre el mismo par de vértices, Floyd-Warshall selecciona la de menor peso correctamente, pero la visualización puede mostrar líneas superpuestas.
4. **Pesos negativos:** Dijkstra no funciona con pesos negativos. En LogísTEC todos los pesos son distancias en metros (enteros positivos), por lo que esta restricción nunca se viola.
5. **Interfaz gráfica con muchas aristas:** con más de 60–70 aristas el panel puede volverse difícil de leer porque los pesos se superponen.

8 Actividades por Estudiante

Integrante	Actividad	Fecha de inicio	Tiempo dedicado (aprox.)
Emanuel Rojas	Implementación útil (LinkedList, MinHeap, Queue, Stack, UnionFind)	28/05	8 horas
Pablo Valverde	Tests unitarios de estructuras de datos	29/05	6 horas

Integrante	Actividad	Fecha de inicio	Tiempo dedicado (aprox.)
Alejandro Arias	Algoritmos: BFS, DFS, Warshall, Dijkstra	25/05	7 horas
Alejandro Arias	Algoritmos: Floyd-Warshall, Prim, Kruskal	26/05	6 horas
Alejandro Arias	Implementación Planner (Best-Fit, NN, MST-based)	26/05	8 horas
Andrés Aguilar	Integración: CityLoader, JsonParser	28/05	6 horas
Pablo Valverde	Interfaz gráfica (MainWindow, GraphPanel)	31/05	9 horas
Andrés Aguilar	Generador del reporte final (resultados de todos los algoritmos)	30/05	3 horas
Pablo Valverde	Caso de prueba JSON	26/05	30 minutos
Todos	Documentación	11/06	10 horas

9 Problemas Encontrados

9.1 Conflicto de nombre: Package vs java.lang.Package

Descripción: Al nombrar la clase de paquetes logísticos `Package`, el compilador producía ambigüedades con `java.lang.Package` (clase de la JDK).

Intentos: Se probó con calificación explícita (`graph.Package`), pero la ambigüedad persistía en ciertos imports.

Solución: Renombrar la clase a `Parcel`, término equivalente en inglés para paquete de envío, sin conflicto con la JDK.

Conclusión: Evitar nombres que colisionen con clases de `java.lang`, que se importan automáticamente en todo archivo Java.

Referencia: JLS 7.5.3 — Single-Type-Import declarations.

9.2 Desbordamiento aritmético en Floyd-Warshall

Descripción: Al sumar `dist[i][k] + dist[k][j]` con valores `Integer.MAX_VALUE/2`, la suma producía desbordamiento y valores negativos, corrompiendo la matriz.

Solución: Convertir a `long` antes de sumar: `long candidate = (long)dist[i][k] + dist[k][j]`, y verificar antes de asignar de vuelta a `int`.

Conclusión: En cualquier algoritmo que acumule sumas sobre valores grandes marcados como INF, usar `long` o doble verificación antes de asignar.

10 Conclusiones y Recomendaciones

1. Implementar las estructuras de datos desde cero obliga a comprender sus invariantes internas. El heap binario demostró ser notablemente eficiente para mantener un mínimo actualizable, pieza fundamental de Dijkstra y Prim.
2. Separar el TDA Grafo de los algoritmos que operan sobre él (paquetes `graph` y `algorithms` independientes) facilitó las pruebas unitarias.
3. Prim y Kruskal producen siempre el mismo costo de MST en grafos con pesos distintos. La verificación empírica de igualdad de costos es una prueba sencilla y robusta de corrección.
4. La heurística MST-based no siempre supera a Nearest Neighbor, pero ofrece garantía teórica de 2-aproximación para métricas de distancias entre puntos.

Recomendaciones:

- Mejorar `decreaseKey` con un heap indexado (mapa de elemento a posición en el arreglo), reduciendo su complejidad de $O(n)$ a $O(\log n)$.
- Para extender a instancias reales, reemplazar el parser JSON propio por Gson únicamente en la capa de E/S, manteniendo el núcleo algorítmico sin dependencias externas.

11 Bibliografía

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.
- Oracle. (2024). *Java Platform, Standard Edition 21 API Specification*. <https://docs.oracle.com/en/java/javase/21/docs/api/>
- Rosenkrantz, D. J., Stearns, R. E., & Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3), 563–581.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley. <https://algs4.cs.princeton.edu/>
- Vargas Rivera, A. (2026). *Lección 06 — Graphs*. Curso CE1103 Algoritmos y Estructuras de Datos I, Tecnológico de Costa Rica.

12 Diagrama de Clases — Justificación del Diseño

[Hacer click aquí para visualizar el diagrama de clases UML](#)

(En caso de que no funcione el enlace, puede visualizarlo como imagen jpg en el repositorio del proyecto)

12.1 Organización en paquetes

Paquete	Responsabilidad
<code>graph</code>	Modelos de dominio puros: Vertex, Edge, Graph, Parcel, Truck. Sin lógica algorítmica.
<code>util</code>	Estructuras de datos genéricas independientes del dominio: LinkedList, MinHeap, Queue, Stack, UnionFind.
<code>algorithms</code>	Algoritmos sobre grafos: recorridos, caminos mínimos, MST. Depende de <code>graph</code> y <code>util</code> , nunca de <code>planner</code> ni <code>ui</code> .
<code>planner</code>	Lógica de negocio: asignación de paquetes y planificación de rutas. Usa <code>algorithms</code> y <code>graph</code> .
<code>io</code>	Entrada/salida: parseo de JSON, carga de la ciudad, generación de reportes.
<code>ui</code>	Vista: ventana Swing y panel de visualización gráfica del grafo.

12.2 Principios de diseño aplicados

- **Separación de responsabilidades (SRP):** cada clase tiene una única razón para cambiar. `FloydWarshall` solo calcula distancias; `Planner` solo toma decisiones lógicas usando esa matriz.
- **Encapsulación:** los campos de todas las clases son privados y se exponen únicamente a través de getters/setters explícitos.
- **Composición sobre herencia:** `Queue` y `Stack` no heredan de `LinkedList`; la contienen. Esto evita exponer métodos inapropiados.
- **Principio de mínima dependencia:** el paquete `algorithms` no importa nada de `planner` ni de `ui`, lo que facilita realizar pruebas unitarias.

13 Bitácora de Trabajo

Fecha	Participantes	Actividades / Resultado
10/05/2026	Todos	Reunión inicial: división de tareas, repositorio Git, estructura de paquetes.
28/05/2026	Emanuel Rojas	Implementación de LinkedList, MinHeap; pruebas con grafos pequeños.
25/05/2026	Alejandro Arias	Implementación de BFS, DFS, Warshall; validación manual con grafo de 5 vértices. Elaboración de caso de prueba mínimo.

Fecha	Participantes	Actividades / Resultado
26/05/2026	Alejandro Arias	Implementación de Dijkstra, Floyd-Warshall; detección del bug de desbordamiento.
27/05/2026	Alejandro Arias y Pablo Valverde	Implementación de Prim y Kruskal; verificación de costos iguales.
27/05/2026	Alejandro Arias y Pablo Valverde	Implementación del Planner: Best-Fit, NN y MST-based.
02/06/2026	Andrés Aguilar	Elaboración de la interfaz gráfica, generador de reporte y json parser.
10/06/2026	Todos	Reunión para detalles en el avance de la documentación y del código final.
12/06/2026	Andrés Aguilar	Elaboración del diagrama de clases UML.
13/06/2026	Emmanuel Rojas y Andrés Aguilar	Modificación de la documentación y migración a formato Latex. Creación del ZIP con el contenido completo del proyecto.